

BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 2: Modern Cryptography

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 31/03/2026

Nhóm 4 – NT101.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24520313	Đỗ Duy Đức		100%
24520330	Trần Minh Đức		100%

GITHUB:

<https://github.com/DucMMT1/NT101.Q22-ATMMT/tree/main>

Cấu trúc Feistel và Sự lan truyền thay đổi

NHIỆM VỤ 2.1(*)

Lý thuyết:

- Feistel là một mô hình thiết kế thuật toán mã hóa đối xứng
- Nguyên lý hoạt động
 - + Plaintext sẽ được chia làm 2 nửa (L và R)
 - + Thực hiện qua nhiều round, với mỗi round thực hiện:

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

Trong đó: F là hàm vòng, K là khóa con

VD: Giả sử ta có $L_0 = 1111$ và $R_0 = 0000$

Bước 1: Lấy R_0 đưa vào hàm F cùng với khóa K_1

-> Kết quả ví dụ: $F(R_0, K_1) = 1010$ (Với hàm F là $00 \rightarrow 10$)

Bước 2: XOR với L_0

-> $L_0 \oplus F = 1111 \oplus 1010 = 0101$

Bước 3: Hoán đổi

$L_1 = R_0 = 0000$

$R_1 = 0101$

Bước 4: việc lặp lại nhiều vòng sẽ cho ra kết quả cuối cùng

Thực hành:

Input:

- M1 = 0xAB = 1010 1011
- M2 = 0xAC = 1010 1100
- Key = 0x12

Output:

- In giá trị (L, R) sau mỗi vòng của cả hai quá trình

Mô tả code:

1. Hàm F (Hàm giả ngẫu nhiên)

```
#Thực hiện XOR giữa right và subkey (R,K)
def F(right, subkey):
    return (right ^ subkey) & 0x0F
```

2. Hàm feistel_round

- Chứa công thức 1 vòng của Feistel

```
def feistel_round(L_in, R_in, subkey):
    #LE_i = RE_{i-1}
    L_out = R_in
    #RE_i = LE_{i-1} XOR F(RE_{i-1}, K_i)
    R_out = L_in ^ F(R_in, subkey)
    return L_out, R_out
```

3. Hàm track_avalanche

- Chia plaintext thành 2 nửa bằng nhau
- Tạo khóa con (subkey) từ khóa gốc key bằng cách dịch bit và cộng dồn
- Vòng lặp 4 lần với mỗi lần gọi hàm feistel_round và in ra giá trị nhị phân
- Ghép 2 nửa 4 bit thành bản cuối cùng

```
def track_avalanche(msg, key):
    # Tách bản rõ thành 2 nửa L và R (mỗi nửa 4 bit)
    L, R = (msg >> 4) & 0x0F, msg & 0x0F

    # Tạo các subkeys
    subkeys = [key & 0x0F, (key >> 4) & 0x0F, (key + 1) & 0x0F, (key + 2) & 0x0F]

    print(f"Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}")

    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, f'0{4}b')}, R={format(R, f'0{4}b')}")
    return (L << 4) | R
```

4. Kết quả:

```
--- Ma hoa M1 ---
Khởi tạo: L=1010, R=1011
Vòng 1: L=1011, R=0011
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 4: L=1001, R=0100
```

```
--- Ma hoa M2 ---
Khởi tạo: L=1010, R=1100
Vòng 1: L=1100, R=0100
Vòng 2: L=0100, R=1001
Vòng 3: L=1001, R=1110
Vòng 4: L=1110, R=0011
```

Nhận xét: Ban đầu 2 plaintext chỉ khác nhau 1 bit, sau quá trình chạy và thực hiện các hàm qua từng vòng thì khiến plaintext khác nhau nhiều bit hơn

Mode of Operation - Chế độ hoạt động

NHIỆM VỤ 2.2:

- AES (Advanced Encryption Standard) là thuật toán mã hóa khối. Tuy nhiên để mã hóa lượng dữ liệu lớn hơn kích thước tiêu chuẩn (128 bit), ta sử dụng “Modes of Operation”.
- + AES-ECB: Chia dữ liệu thành các khối cố định, và mã hóa độc lập mỗi khối với cùng một khóa K.
 - => Khối bản rõ P1 mã hóa thành C1, P2 mã hóa thành C2, nếu $P1 = P2$ thì $C1 = C2$.
- + AES-CBC: Trước khi mã hóa, mỗi khối bản rõ sẽ được thực hiện phép XOR với khối bản mã của vòng ngay trước đó.
 - => Khối đầu tiên (P1) thực hiện XOR với vector khởi tại (IV) trước khi mã hóa.
 - => Khối thứ hai (P2) thực hiện XOR với bản mã của khối thứ nhất (C1) trước khi mã hóa.
 - => Lặp lại.

Nhận xét:

```
Plaintext: UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_
-----
ECB Ciphertext (Hex): 02b0647de210da452e3bdcadc415814e02b0647de210da452e3bdcadc415814e
Block 1 (ECB): 02b0647de210da452e3bdcadc415814e
Block 2 (ECB): 02b0647de210da452e3bdcadc415814e
=> Nhận xét: Hai khối bản mã GIỐNG HẾT NHAU
-----
CBC Ciphertext (Hex): 47747530a46ebd156c1da8763c54be1b46807a7ec513e5e69d9256881a0f0c09
Block 1 (CBC): 47747530a46ebd156c1da8763c54be1b
Block 2 (CBC): 46807a7ec513e5e69d9256881a0f0c09
=> Nhận xét: Hai khối bản mã KHÁC NHAU
```

- AES-ECB:

+ Kết quả cho thấy hai bản mã giống nhau

=> ECB mã hóa độc lập từng khối dữ liệu, do đó đầu vào giống nhau sẽ luôn cho ra đầu ra giống nhau => Bảo mật thấp => Người tấn công có thể nhận biết các mẫu lặp lại trong thông tin mà không cần giải mã.

- AES-CBC:

+ Kết quả hai bản mã khác nhau

=> Mỗi khối bản rõ được XOR với bản mã của khối trước đó => Khối 1 XOR với IV, Khối 2 XOR với bản mã khối 1 => Chaining này làm thay đổi đầu vào của hàm mã hóa cho từng khối => Bảo mật cao => Che giấu hoàn toàn các mẫu lặp lại của dữ liệu gốc.

Avalanche Effect

NHIỆM VỤ 2.3:

- Hiệu ứng thác đổ (Avalanche Effect): một sự thay đổi nhỏ ở đầu vào gây ra sự thay đổi đáng kể ở đầu ra.

```

--- Ket qua voi Key (MSSV): 24520330 ---
Plaintext 1: STAYHOME -> Ciphertext (Hex): 32f18d564bf57462
Plaintext 2: STAYHOMA -> Ciphertext (Hex): caf71dc64589a4f0
Hamming Distance: 25 bits
Avalanche Ratio: 39.06%

--- Ket qua voi Key (MSSV): 24520313 ---
Plaintext 1: STAYHOME -> Ciphertext (Hex): 0a9783021a609e7b
Plaintext 2: STAYHOMA -> Ciphertext (Hex): a2d6c71955d4160d
Hamming Distance: 27 bits
Avalanche Ratio: 42.19%
  
```

- Hai chuỗi “STAYHOME” và “STAYHOMA” khác nhau 1 bit kí tự nhưng bản mã thu được thay đổi tận 39% và 42% => Chứng minh thuật toán có khả năng khuếch tán tốt, một thay đổi nhỏ cũng lan tỏa ra toàn bộ bản mã.

- Khi thay đổi khóa (MSSV):

+ Bản mã của chuỗi “STAYHOME” thay đổi hoàn toàn, tỉ lệ thác đổ cũng tăng lên khoảng 3% => Mỗi bit đóng vai trò quan trọng trong việc xáo trộn dữ liệu, không có mối liên hệ tuyến tính đơn giản giữa bản mã và khóa.

- Tỉ lệ thác đổ khá cao cho thấy độ an toàn của thuật toán, đảm bảo các tính chất an toàn cơ bản của một hệ mật mã về mặt cấu trúc.

NHIỆM VỤ 2.4:

Phần 1: Khởi tạo các giá trị:

Chuẩn bị dữ liệu đầu vào hợp lệ cho thuật toán AES-128, vốn yêu cầu dữ liệu phải là bội số của 16 byte (block size). Sử dụng `os.urandom()` để sinh ngẫu nhiên 1000 byte.

```

BLOCK_SIZE = 16 # 16 bytes (128-bit)
SECRET_KEY = b'1234567890123456' # Khóa 128-bit
INIT_VECTOR = b'abcdefghijklmnp' # IV 128-bit
  
```

Phần 2: Mã hóa và gây lỗi

Tạo ra bản mã và mô phỏng hiện tượng nhiễu bit (bit flip) xảy ra trong quá trình truyền tải dữ liệu thực tế. Khởi tạo đối tượng cipher theo từng chế độ (ECB, CBC, CFB, OFB) để mã hóa dữ liệu. Tiếp theo, sử dụng toán tử bitwise XOR ($\wedge = 1$) tác động trực tiếp vào vị trí byte thứ 26 (tức index 25) của bản mã để đảo ngược đúng 1 bit.

- Hàm `flip_bit`

```
# LỖI TRÊN CIPHERTEXT
# Đảo 1 bit tại vị trí byte_index (Mặc định byte 26 - index 25)
def flip_bit(cipher_bytes: bytearray, byte_index=25):
    cipher_bytes[byte_index] ^= 1
    return cipher_bytes
```

- Hàm create_cipher:

```
# KHỞI TẠO CIPHER
def create_cipher(mode: str):
    if mode == 'ECB':
        return AES.new(SECRET_KEY, AES.MODE_ECB)

    elif mode == 'CBC':
        return AES.new(SECRET_KEY, AES.MODE_CBC, INIT_VECTOR)

    elif mode == 'CFB':
        return AES.new(SECRET_KEY, AES.MODE_CFB, INIT_VECTOR, segment_size=128)

    elif mode == 'OFB':
        return AES.new(SECRET_KEY, AES.MODE_OFB, INIT_VECTOR)

    else:
        raise ValueError(f"Mode không hợp lệ: {mode}")
```

Phần 3: Giải mã và thống kê

Đánh giá mức độ tàn phá của 1 bit lỗi trên bản mã đối với toàn bộ bản rõ sau khi giải mã. Dùng bản mã đã hỏng đưa qua hàm giải mã. Sau đó, cắt bản rõ vừa giải mã và bản rõ gốc thành từng đoạn 16 byte (block-by-block) để so sánh chéo. Nếu có bất kỳ sự khác biệt nào (!=), biến đếm lỗi sẽ tăng lên.

- Hàm simulate_error:


```
# MÔ PHỎNG LAN TRUYỀN LỖI
def simulate_error(mode: str):
    """
    Mô phỏng:
    1. Sinh dữ liệu & Padding
    2. Mã hóa
    3. Gây lỗi 1 bit
    4. Giải mã
    5. Đếm block lỗi
    """

    # 1. Sinh dữ liệu 1000 byte
    raw_plaintext = generate_random_data()
    plaintext = pad(raw_plaintext, BLOCK_SIZE)

    # 2. Mã hóa bằng AES-128
    encrypt_cipher = create_cipher(mode)
    ciphertext = bytearray(encrypt_cipher.encrypt(plaintext))

    # 3. Làm hỏng bản mã bằng cách đảo 1 bit tại byte thứ 26
    corrupted_cipher = flip_bit(ciphertext)

    # 4. Giải mã bản mã đã bị lỗi
    decrypt_cipher = create_cipher(mode)
    decrypted_text = decrypt_cipher.decrypt(corrupted_cipher)

    # 5. Đếm block lỗi với chế độ mã hóa tương ứng
    error_blocks = count_corrupted_blocks(plaintext, decrypted_text)

    print(f"[{mode}] Số block bản rõ bị hỏng: {error_blocks}")
```

- Hàm count_corrupted_blocks

```
# ĐẾM BLOCK BỊ LỖI
# So sánh từng block 16 byte để đếm block bị sai
def count_corrupted_blocks(original: bytes, decrypted: bytes):
    corrupted = 0
    for i in range(0, len(original), BLOCK_SIZE):
        original_block = original[i:i + BLOCK_SIZE]
        decrypted_block = decrypted[i:i + BLOCK_SIZE]

        if original_block != decrypted_block:
            corrupted += 1
    return corrupted
```

Phần 4: Nhận xét

Xây dựng script bằng Python, lần lượt chạy dữ liệu qua 4 chế độ (ECB, CBC, CFB, OFB). Tại mỗi chế độ, tiến hành mã hóa bằng AES-128, chủ động làm hỏng 1 bit tại byte thứ 26 của bản mã, sau đó giải mã và thống kê số khối dữ liệu bản rõ bị hỏng.

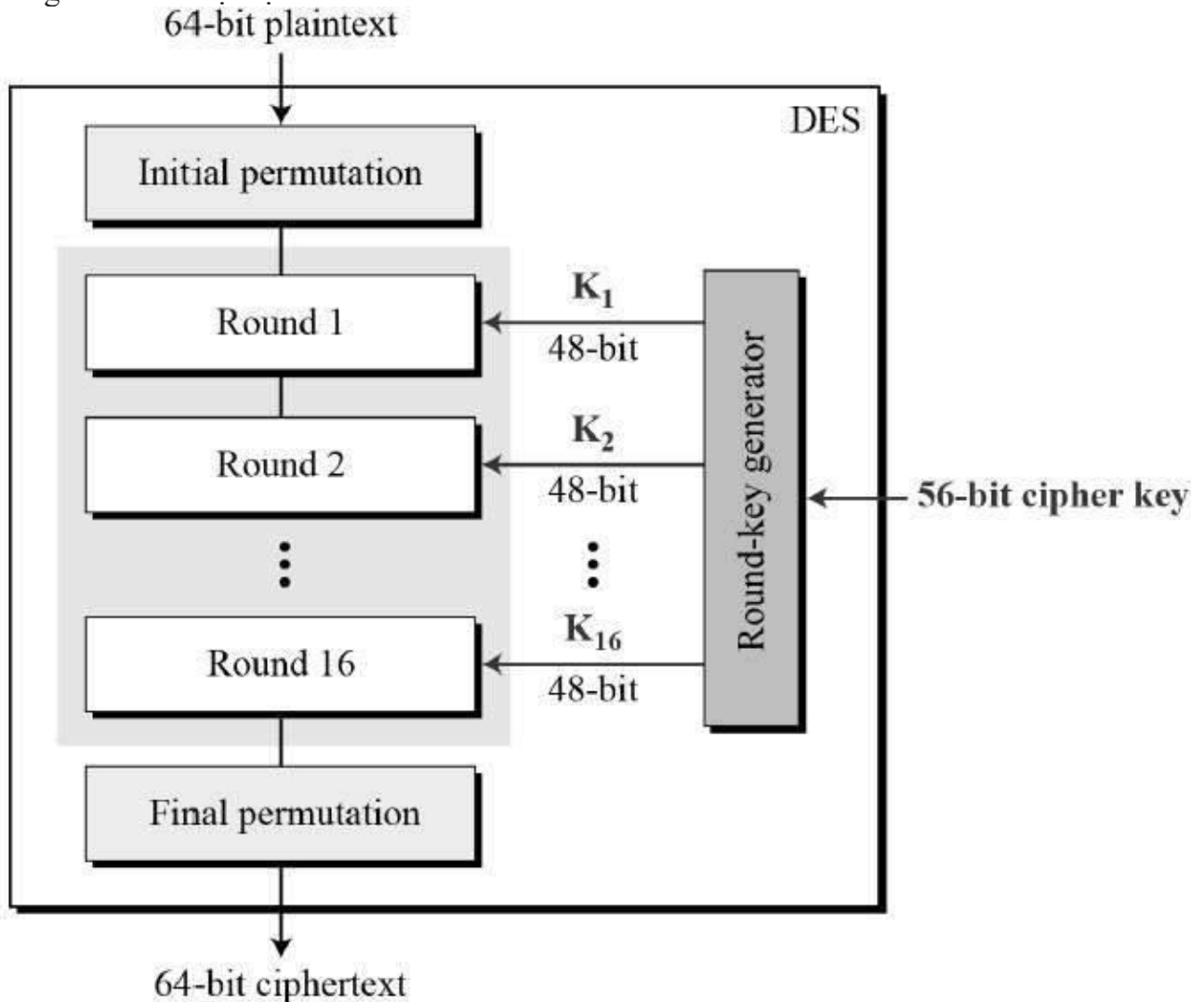
- ECB và OFB: Chỉ có 1 khối bị hỏng. Lỗi được cô lập, không ảnh hưởng đến các khối phía sau.
- CBC và CFB: Có 2 khối bị hỏng. Một bit sai lệch ở khối này đã lan truyền và phá hỏng quá trình giải mã của khối tiếp theo.

NHIỆM VỤ 2.5:

- DES (Data Encryption Standard) hay còn gọi là tiêu chuẩn mã hóa dữ liệu, là thuật toán mã hóa đối xứng khối. Dựa trên cấu trúc mật mã Feistel (gồm 16 vòng luân phiên các phép

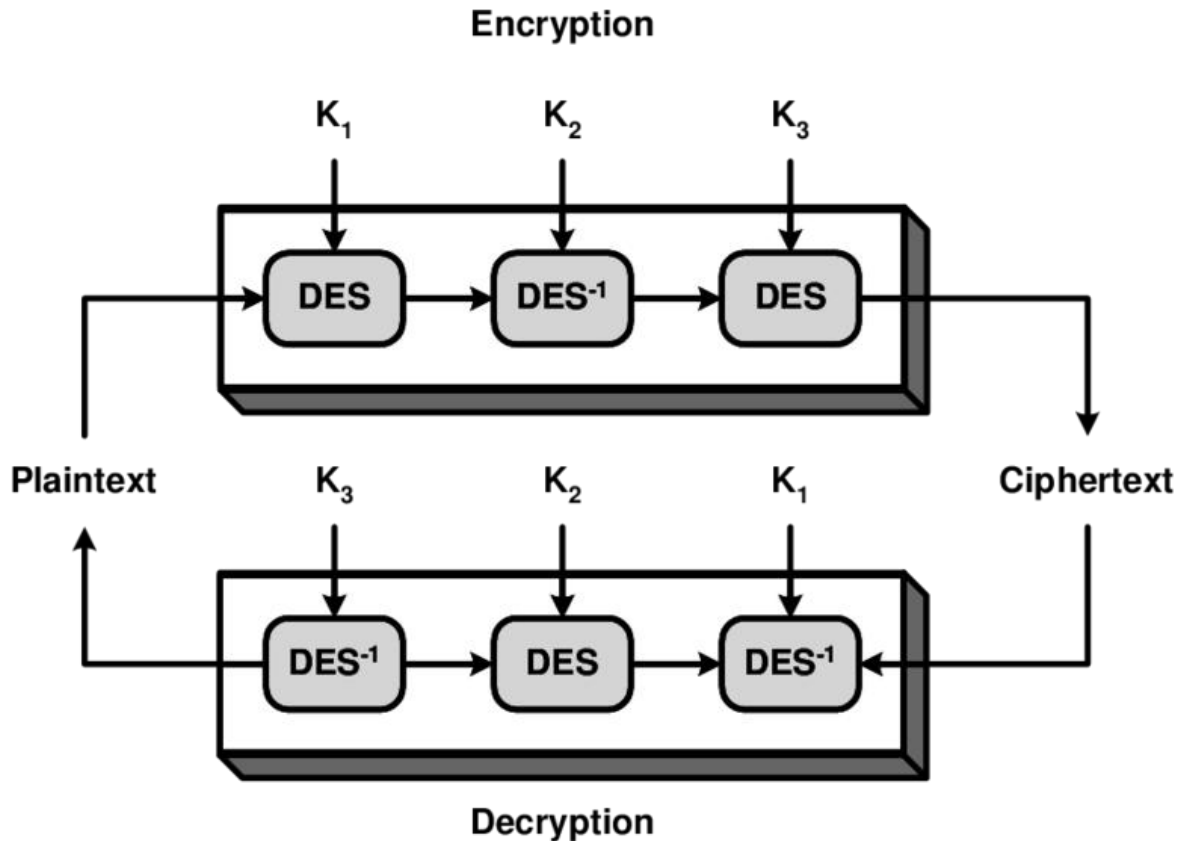
thế và hoán vị). Mật mã này chia bản rõ thành hai nửa để xử lý, mã hóa một khối đầu vào có độ dài 64 bit bằng một khóa có độ dài 56 bit.

DES hiện nay rất yếu và không còn an toàn để sử dụng. Với chiều dài khóa chỉ 56 bit, kẻ tấn công có thể dễ dàng phá vỡ thuật toán này bằng phương pháp vét cạn (brute-force) với phần cứng tính toán hiện tại.



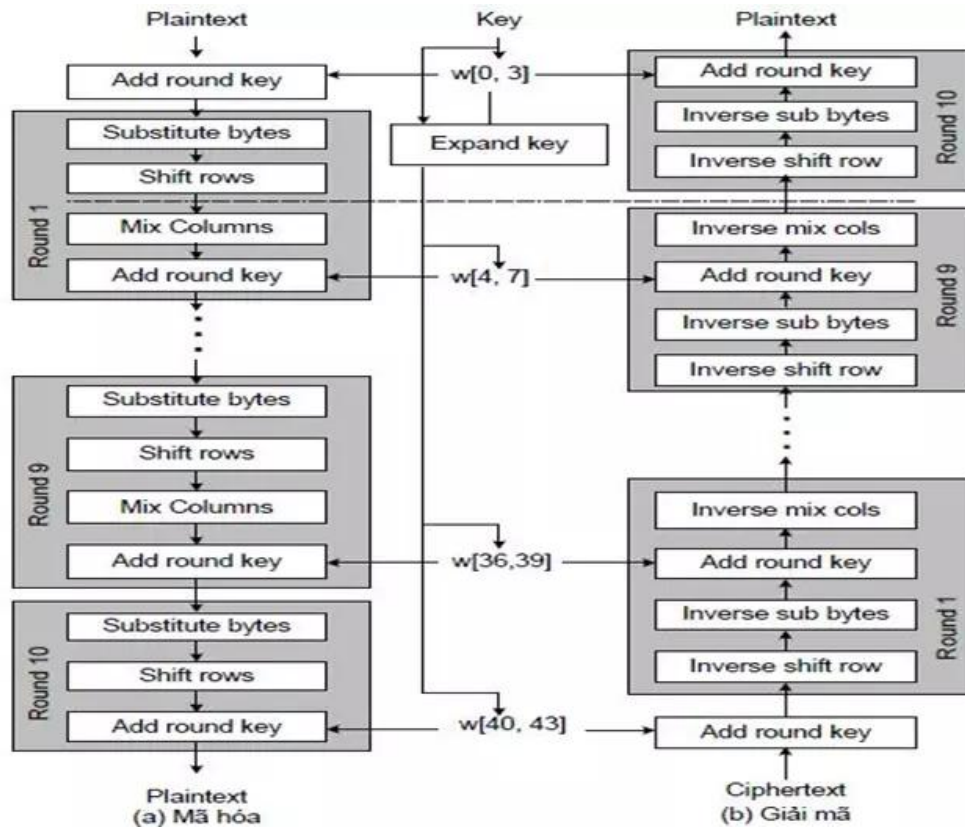
- 3DES (Triple DES) là thuật toán mã hóa khối nâng cao bằng cách áp dụng quy trình DES gốc ba lần liên tiếp lên dữ liệu, sử dụng 1, 2 hoặc 3 khóa riêng biệt (mỗi khóa 56-bit) để tăng cường bảo mật. Quy trình hoạt động chính là Mã hóa (Encrypt) - Giải mã (Decrypt) - Mã hóa (Encrypt) để đảm bảo tính tương thích ngược và nâng cao độ an toàn. Kế thừa nguyên vẹn cấu trúc của DES nhưng áp dụng thuật toán mã hóa này 3 lần liên tiếp trên mỗi khối dữ liệu 64-bit (thường theo quy trình: Mã hóa bằng khóa 1 - Giải mã bằng khóa 2 - Mã hóa bằng khóa 3). Quá trình này giúp tăng không gian khóa lên 112 hoặc 168 bit tùy thuộc vào số lượng khóa độc lập được sử dụng.

3DES: An toàn hơn DES đáng kể nhờ không gian khóa lớn, nhưng lại gặp vấn đề nghiêm trọng về hiệu năng (chạy chậm gấp 3 lần DES do phải lặp lại thuật toán). Ngoài ra, kích thước khối 64-bit vẫn tiềm ẩn rủi ro bị tấn công va chạm (collision attacks) nếu áp dụng mã hóa trên khối lượng dữ liệu lớn.



- AES (Advanced Encryption Standard) là thuật toán mã hóa khối đối xứng, sử dụng cùng một khóa (128, 192 hoặc 256-bit) để mã hóa và giải mã dữ liệu 128-bit. Nó hoạt động qua nhiều vòng lặp (10, 12, hoặc 14 vòng) bao gồm các phép biến đổi: SubBytes (thay thế), ShiftRows (dịch chuyển), MixColumns (trộn), và AddRoundKey (cộng khóa) để tạo ra mã hóa bảo mật cao.

AES có độ mạnh cực kỳ cao, chống lại hoàn toàn các cuộc tấn công vét cạn và thám mã hiện đại. Đồng thời, AES được thiết kế để tối ưu hóa tài nguyên, cho tốc độ mã hóa rất nhanh và ổn định trên cả môi trường phần cứng lẫn phần mềm.



	DES	3DES	AES
Độ dài khóa	56 bits	112 hoặc 168 bits	128, 192 hoặc 256 bits
Độ lớn Block	64 bits	64 bits	128 bits
Tốc độ	Chậm	Chậm	Nhanh
Bảo mật	Không đủ	Không đủ	Rất mạnh
Cấu trúc dữ liệu	Feistel	Feistel	Substitution Permutation
Thời gian cần thiết để kiểm tra tất cả các khóa có thể có với tốc độ 50 tỷ khóa mỗi giây.	400 ngày	800 ngày	5×10^{21} ngày

Tại sao không nên sử dụng mã hóa double-DES (2DES)?

Việc chạy thuật toán DES 2 lần liên tiếp (2DES) với 2 khóa K1 và K2 hoàn toàn độc lập tưởng chừng sẽ giúp tăng gấp đôi độ dài khóa thành 112 bit ($56 + 56$), từ đó nhân đôi độ mạnh bảo mật của hệ thống. Tuy nhiên, trong thực tế, 2DES hoàn toàn vô dụng trước Tấn công gặp gỡ ở giữa (Meet-in-the-Middle Attack):

- Thay vì phải vét cạn toàn bộ không gian khóa 2^{112} , kẻ tấn công tính toán bản rõ mã hóa bằng khóa K1 từ một đầu, đồng thời lấy bản mã tiến hành giải mã bằng khóa K2 từ đầu kia, sau đó tìm kiếm và đối chiếu kết quả trung gian trùng khớp ở giữa.
- Phương pháp này làm giảm độ phức tạp của cuộc tấn công xuống chỉ còn tương đương 2^{57} thao tác tìm kiếm.
- Nghĩa là, 2DES tốn gấp đôi thời gian và tài nguyên tính toán so với DES nhưng mức độ bảo mật gần như không hề được cải thiện (chỉ tăng từ 2^{56} lên 2^{57}).

Mersenne Prime - Số nguyên tố lớn

NHIỆM VỤ 2.6(*):

Lý thuyết:

- Thuật toán Miller-Rabin là cách kiểm tra nhanh một số có phải số nguyên tố hay không (mang tính xác suất).

+Ý tưởng thuật toán :Thay vì kiểm tra chia hết từng số, ta lợi dụng một tính chất của số nguyên tố liên quan đến lũy thừa modulo. Cụ thể, ta viết $n-1$ dưới dạng $2^s \cdot d$ với d là số lẻ, rồi chọn ngẫu nhiên một số a . Sau đó tính $a^d \bmod n$ và bình phương lặp lại nhiều lần. Nếu trong quá trình đó kết quả rơi vào các giá trị như 1 hoặc $n-1$, thì n có khả năng là số nguyên tố. Ngược lại, nếu không thỏa các điều kiện này, thì chắc chắn n là hợp số. Bằng cách thử với nhiều giá trị a khác nhau, ta giảm xác suất sai xuống rất nhỏ, nên trong thực tế có thể coi kết quả là gần như chính xác.

- Thuật toán Euclid là cách tìm ước chung lớn nhất(gcd) của hai số. Ý tưởng của nó là: ước chung của hai số sẽ không thay đổi nếu ta thay số lớn bằng phần dư khi chia cho số nhỏ hơn. Vì vậy, ta cứ lặp lại việc lấy phần dư của phép chia, rồi thay cặp số ban đầu bằng cặp mới (số nhỏ hơn và phần dư). Quá trình này tiếp tục cho đến khi phần dư bằng 0, và khi đó số còn lại chính là ước chung lớn nhất.

- Phép nhân module có tính chất sau:

$$(a \times b) \equiv [(a \bmod n) \times (b \bmod n)] \bmod n$$

Tính chất này có thể được áp dụng để tính toán lũy thừa module (modular exponentiation) trong trường hợp có số mũ lớn.

THỰC HÀNH

Input:

- Kích thước bit cần tạo số nguyên tố (8, 16, 64 bit).
- Số nguyên tố Mersenne thứ 10 làm mốc giới hạn trên.
- Hai số nguyên lớn để tính GCD.
- Cơ số $a = 7$, số mũ $x = 40$ và module $p = 19$ cho phép tính lũy thừa

Output:

- Các số nguyên tố ngẫu nhiên có độ dài bit tương ứng.
- Danh sách 10 số nguyên tố lớn nhất nhỏ hơn $2^{89} - 1$
- Kết quả Ước chung lớn nhất (GCD) của hai số.
- Kết quả của phép toán $7^{40} \bmod 19$.

Mô tả code:

- Hàm `is_prime_miller_rabin` (Thuật toán Miller-Rabin)

```
def is_prime_miller_rabin(n, k=5):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False

    r = n - 1
    d = 0
    while r % 2 == 0:
        d += 1
        r //= 2

    # Bắt đầu k vòng lặp thử nghiệm
    for _ in range(k):
        a = random.randint(2, n - 2) # Chọn ngẫu nhiên cơ số a trong khoảng [2, n-2]

        # Tính x = (a^r) mod n
        x = modular_exponentiation(a, r, n)

        if x == 1 or x == n - 1:
            continue # Nếu thỏa mãn, chuyển sang thử nghiệm a tiếp theo

        for _ in range(d - 1):
            # Tính tiếp x = (x^2) mod n
            x = modular_exponentiation(x, 2, n)
            if x == n - 1:
                break # Thỏa mãn, chuyển sang thử nghiệm a tiếp theo
        else:
            # Nếu vòng lặp trong (for _ in range(d-1)) chạy hết mà không break, n là hợp số
            return False
```

- Hàm generate n bit prime: Tạo ra một số nguyên tố ngẫu nhiên có độ dài bit xác định.

```
#Tạo ra một số nguyên tố ngẫu nhiên có độ dài bit xác định.
def generate_n_bit_prime(bits):
    while True:
        # Sinh một số ngẫu nhiên có 'bits' bit
        num = random.getrandbits(bits)

        # Đảm bảo bit cao nhất là 1 (đủ số bit) và bit thấp nhất là 1 (để là số lẻ)
        num |= (1 << bits - 1) | 1

        if is_prime_miller_rabin(num):
            return num
```

- Hàm gcd (Thuật toán Euclid):

```
def gcd(a, b):
    while b != 0:
        # Cập nhật a bằng b, và b bằng phần dư của a chia b
        a, b = b, a % b
    return a
```

- Hàm modular_exponentiation (Lũy thừa Module):

```
def modular_exponentiation(a, x, p):  
    result = 1  
    a = a % p  
  
    while x > 0:  
        # Nếu bit cuối cùng của x là 1 (nghĩa là x lẻ)  
        if x % 2 == 1:  
            result = (result * a) % p # Nhân cơ số a vào kết quả  
  
        # Dịch phải x đi 1 bit (= chia 2)  
        x //= 2  
  
        a = (a * a) % p  
  
    return result
```

HẾT